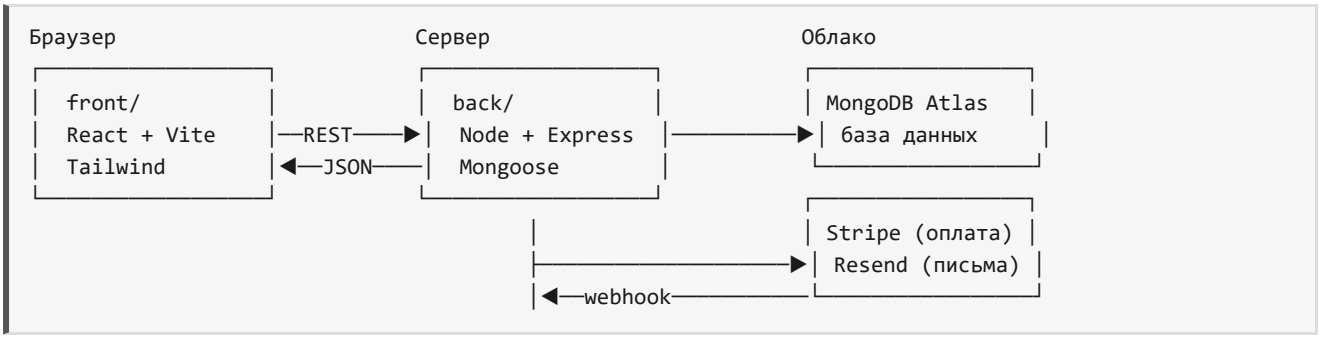
DARIN'S HANDMADE — как устроен код

Шпаргалка для защиты. Читается сверху вниз: сначала общая картина, потом файлы, потом сквозные сценарии, в конце — вероятные вопросы с ответами.

1. Общая картина за минуту

Проект состоит из двух отдельных приложений, которые общаются по HTTP:



Ключевая мысль: фронт ничего не знает о базе. Он только шлёт HTTP-запросы на `/api/...` и рисует то, что вернул сервер. Сервер — единственный, кто ходит в базу.

Архитектура бэкенда — три слоя. Так учили на курсе, и так здесь сделано:

```
routes/       — какой адрес какой функции достаётся
controllers/  — что делать: проверить, сходить в базу, ответить
models/       — как данные выглядят в базе (схемы Mongoose)
```

Плюс `middleware/` — то, что выполняется **между** маршрутом и контроллером (проверка токена, роли, загрузка файла, валидация).

2. Путь одного запроса

Пример: менеджер удаляет товар. Запрос `DELETE /api/products/123`.

Шаг	Где	Что происходит
1	index.js	`app.use('/api/products', productsRoute)` — узнал префикс, отдал роутеру
2	productsRoute.js	`router.delete('/:id', protect, restrictTo(...), deleteProduct)`
3	authMiddleware.js	`protect` читает заголовок `Authorization: Bearer ...`, проверяет токен, кладёт пользователя в `req.user`
4	roleMiddleware.js	`restrictTo('manager','admin')` смотрит `req.user.role`; не подходит — `403` и дальше не пускает
5	ProductController.js	`deleteProduct` удаляет запись, стирает файл картинки, отвечает `{ message }`

Каждый `middleware` либо вызывает `next()` (пропустить дальше), либо сам отвечает и цепочка обрывается. Это и есть ответ на вопрос «что такое middleware».

3. Бэкенд по файлам

Точка входа

back/index.js — собирает приложение. Важен **порядок**, он не случайный:

1. `app.set('trust proxy', 1)` — сайт стоит за прокси хостинга, без этого лимитер считал бы все запросы как с одного IP.
2. `app.use(cors())` — разрешает браузеру ходить с одного домена (фронт) на другой (сервер).
3. **Вебхук Stripe — до `express.json()`.** Stripe подписывает сырое тело запроса; если его сначала разобрать в объект, подпись не сойдётся. Поэтому только у этого маршрута `express.raw()`.
4. `express.json()` — разбирает JSON во всех остальных запросах в `req.body`.
5. `helmet()` — проставляет защитные HTTP-заголовки.
6. Лимитеры: 100 запросов / 15 мин на весь `/api/`, 10 / 15 мин на вход и регистрацию, 5 / час на запрос сброса пароля.
7. `/uploads` — раздача картинок товаров как статических файлов.
8. Четыре роутера.
9. Обработчик ошибок — последним, четыре аргумента `(err, req, res, next)`. Нужен, чтобы клиент всегда получал JSON, а не HTML-страницу ошибки.
10. `start()` — сначала соединение с базой, только потом `app.listen`.

back/db/connect.js — три строки, `mongoose.connect(url)`. Вынесено отдельно, чтобы `index.js` не знал деталей подключения.

Модели — `back/models/`

Файл	Что описывает	Заметное
`userModel.js`	пользователь	`password` с `select: false` — не приходит в обычных запросах; `role` (`user` / `manager`); поля сброса пароля
`productModel.js`	товар	`category` — `enum: ['bags','bracelets']`; `stock: null` означает «не отслеживаем остаток»
`cartModel.js`	корзина	`user` + массив `items` с `ref: 'Product'` — по этой ссылке работает `populate`
`orderModel.js`	заказ	`orderNumber`, вложенные `customer` и `shipping`, `deliveryMethod: delivery/pickup`, шесть статусов, блок `payment`

`{ timestamps: true }` во всех — Mongoose сам добавляет `createdAt` и `updatedAt`.

Middleware — `back/middleware/`

- **`authMiddleware.js`** (`protect`) — проверяет JWT. Важная деталь: роль берётся **из базы**, а не из токена. Если менеджера понизили, его старый токен не даст прав.
- **`roleMiddleware.js`** (`restrictTo`) — функция, возвращающая функцию: `restrictTo('manager')` создаёт middleware с уже зашитым списком ролей.

- **`upload.js`** — multer. Кладёт картинку в `uploads/`, имя = очищенное исходное + `Date.now()`, чтобы одинаковые имена не затирали друг друга. Пропускает только `image/*`, лимит 8 МБ.
- **`validate.js`** — проверка форм до контроллера: длина имени, формат почты, длина пароля. Возвращает уже очищенные значения.

Контроллеры — `back/controllers/`

Файл	Функции	Что делает
`AuthController.js`	`signup`, `signin`, `forgotPassword`, `resetPassword`, `getMe`	пароли через `bcrypt`, токен на 1 час
`ProductController.js`	CRUD товаров	`toArray` превращает строку `"Pink, White"` в массив; удаляет старую картинку при замене
`CartController.js`	`getCart`, `addToCart`, `removeFromCart`	`populate('items.product')` подставляет полные товары вместо ссылок
`OrderController.js`	`createCheckoutSession`, `stripeWebhook`, `getAllOrders`, `getStats`, `updateOrderStatus`, `getOrderById`, `getOrderBySession`	самый сложный файл, разбор ниже

Формат ответов одинаковый везде:

```
res.status(200).json({ count: products.length, data: products });  // список
res.status(200).json({ data: product });                           // один объект
res.status(400).send({ message: 'Invalid credentials' });           // ошибка
```

back/utils/email.js — один отправитель писем на весь проект (подтверждение заказа и ссылка сброса пароля). Без переменных окружения письмо просто пишется в консоль — сайт продолжает работать.

4. Фронтенд по файлам

main.jsx — точка входа. Оборачивает приложение в `BrowserRouter`, поэтому `Link` и `useNavigate` работают везде, включая шапку.

App.jsx — все маршруты и охрана страниц менеджера:

```
<Route path='/admin'
  element={isManager ? <ManagerProductsPage token={token} /> : <Navigate to='/login' replace />} />
```

Здесь же `useEffect`, который при каждой смене токена спрашивает `/api/auth/me`. Роль берётся с сервера, а не из `localStorage` — иначе её можно было бы подделать через инструменты разработчика.

api.jsx — единственное место, где записан адрес сервера:

- `API_BASE_URL` — `localhost` на твоей машине, задеплоенный адрес где угодно ещё;
- `api` — экземпляр axios, все запросы идут через него;

- `imageUrl(file_name)` — собирает полный адрес картинки из `uploads`;
- `authHeader(token)` — заголовок `Authorization` для защищённых запросов.

context/CartContext.jsx — корзина. Контекст нужен, чтобы любая страница читала корзину без передачи через пропсы. Две ветки: вошедший пользователь — корзина на сервере, гость — в `localStorage`. При первом входе гостевая корзина переносится на сервер и стирается.

Страницы — `front/src/pages/`:

Папка	Страница
`home/`	главная, собрана из четырёх секций
`beaded-bags/`, `bracelets/`	каталоги, фильтруют общий список по `category`
`product/`	карточка товара, `id` берётся из адреса через `useParams`
`custom-studio/`	конструктор сумки
`cart/`, `checkout/`, `order/`	корзина, оформление, страница «заказ принят»
`login/`	вход, регистрация, забыли пароль, сброс пароля
`admin/`	товары менеджера и заказы менеджера
`legal/`	политика и условия

Стиль кода одинаковый во всех файлах:

```
const HomePage = () => { ... };
export default HomePage;
```

Стрелочная функция, `export default` внизу. Запросы — через `api` с `.then()/.catch()`.

5. Сквозные сценарии

Регистрация и вход

1. `LoginPage` шлёт `POST /api/auth/signup`.
2. `validateSignup` проверяет поля.
3. `signup` хеширует пароль `bcrypt`, создаёт пользователя, подписывает JWT.
4. Фронт кладёт токен и пользователя в `localStorage`, обновляет состояние в `App`.
5. Менеджера перебрасывает на `/admin`, обычного — на главную.

Каталог

`BeadedBagsPage` при открытии делает `GET /api/products`, фильтрует `category === 'bags'`, рисует карточки. Картинка — `imageUrl(bag.file_name)`.

Корзина

- **Гость:** товар кладётся в `localStorage`, ключ `guestCart`.
- **Вошедший:** `POST /api/cart` → `addToCart` находит корзину по `req.user.id`, увеличивает количество или добавляет строку.

- **Вход с непустой гостевой корзиной:** `useEffect` в `CartContext` видит появление токена, отправляет накопленные товары на сервер, чистит `localStorage`.

Конструктор сумки

Три шага: **размер** → **ремешок** → **цвет**.

- Цена: `размер + ремешок` (например Small 180 + Beaded 45 = 225).
- 15 цветов, у каждого фотография бусин вместо цветного кружка.
- Готовая сумка — не товар из каталога, у неё нет `_id`. Она живёт в корзине этого браузера с полем `config`, где записаны размер, ремешок, цвет.

Оформление и оплата

1. `CheckoutPage` шлёт `POST /api/orders/checkout-session` — товары, контакты, адрес, способ получения.
2. `buildOrderItems` по каждой позиции идёт в базу: если `id` настоящий — берёт цену **с сервера**, а не из запроса. Цену сумки из конструктора сервер считает сам по таблицам `SIZE_PRICES` и `STRAP_PRICES`. Это защита: иначе цену можно было бы подменить в браузере.
3. Там же проверяется остаток: «осталось только 2 штуки» — заказ не создаётся.
4. Создаётся заказ со статусом `pending`, в Stripe открывается сессия оплаты, в её `metadata` кладётся `orderId`.
5. Клиент уходит на страницу Stripe. Сервер отвечает только `{ url }`.
6. **Stripe платит — и присылает вебхук** `POST /api/orders/webhook`. Сервер проверяет подпись, находит заказ по `metadata.orderId`, ставит `paid`, **списывает остаток**, отправляет письма покупателю и менеджеру.

Почему статус меняет вебхук, а не браузер: пользователь может закрыть вкладку после оплаты. Вебхук приходит от Stripe напрямую на сервер и не зависит от клиента.

Восстановление пароля

1. `POST /api/auth/forgot-password` с почтой.
2. Генерируется случайный токен; в базу пишется **только его SHA-256 хеш** и срок в один час.
3. На почту уходит ссылка с самим токеном.
4. `POST /api/auth/reset-password` ищет пользователя по хешу токена и непросроченному сроку, ставит новый пароль, обнуляет токен.

Ответ всегда одинаковый — «если такой аккаунт есть, письмо отправлено». Иначе через эту форму можно было бы перебором узнать, кто зарегистрирован.

Менеджер

`ManagerOrdersPage` тянет `GET /api/orders` с фильтрами и `GET /api/orders/stats` для сводки, меняет статус через `PATCH /api/orders/:id/status`. Все эти маршруты закрыты связкой `protect` + `restrictTo`.

6. Быстрый поиск: «где это?»

Вопрос	Файл
Где подключается база	`back/db/connect.js`, вызывается в `back/index.js`
Где проверяется токен	`back/middleware/authMiddleware.js`
Где проверяется роль	`back/middleware/roleMiddleware.js`
Где хешируется пароль	`back/controllers/AuthController.js`
Где загружается картинка	`back/middleware/upload.js`
Где считается цена сумки	`front/src/pages/custom-studio/CustomStudioPage.jsx`
Где списывается остаток	`stripeWebhook` в `back/controllers/OrderController.js`
Где адрес сервера	`front/src/api.jsx`
Где список маршрутов	`front/src/App.jsx`
Где логика корзины	`front/src/context/CartContext.jsx`

7. Вероятные вопросы и ответы

Что такое middleware и зачем? Функция между маршрутом и контроллером. Получает `(req, res, next)`. Либо пропускает дальше через `next()`, либо отвечает сама и обрывает цепочку. У нас так сделаны проверка токена, проверка роли, загрузка файла и валидация форм.

Почему JWT, а не сессии? Сервер не хранит состояние: вся информация в подписанном токене у клиента. Это позволяет держать фронт и бэкенд на разных доменах и легко масштабировать. Токен живёт час.

Токен можно подделать? Нет. Он подписан секретом, который есть только на сервере. Изменишь что-то внутри — подпись не сойдётся и `jwt.verify` бросит ошибку. Но содержимое токена **читается** любым, поэтому пароль туда не кладут.

Где хранится пароль? Только хеш `bcrypt`, исходный пароль нигде не сохраняется. При входе `bcrypt.compare` хеширует введённый и сравнивает с сохранённым. Плюс в схеме стоит `select: false` — пароль не приходит в обычных запросах.

Что такое `populate`? В корзине хранятся только ссылки на товары (`ObjectId`). `populate('items.product')` просит Mongoose подставить вместо ссылок сами документы товаров, чтобы фронт получил название и цену за один запрос.

Что если два человека купят последний товар? Остаток проверяется при создании заказа, а списывается в вебхуке атомарно:

```
Product.updateOne({ _id: id, stock: { $gte: qty } }, { $inc: { stock: -qty } })
```

Условие `stock >= qty` — часть самого запроса, поэтому база не даст уйти в минус: второму просто не найдётся подходящего документа.

Почему цена берётся с сервера, а не из корзины? Всё, что приходит из браузера, можно подменить. `buildOrderItems` по `id` вытаскивает цену из базы и считает сумму сам.

Почему вебхук зарегистрирован до `express.json()`? Stripe подписывает сырое тело. Если `express.json()` разберёт его в объект, восстановить исходные байты уже нельзя и проверка подписи провалится.

Зачем `trust proxy`? Хостинг держит приложение за прокси. Без этой настройки Express видит IP прокси вместо IP клиента, и ограничитель запросов считал бы всех посетителей за одного.

Почему axios вынесен в отдельный файл? Адрес сервера записан в одном месте. При переезде меняется одна строка, а не двадцать файлов. Там же собраны заголовок авторизации и построение адреса картинок.

Зачем контекст для корзины? Корзина нужна и шапке (счётчик), и каталогу, и странице корзины. Без контекста пришлось бы передавать её пропсами через все промежуточные компоненты.

8. Что честно сказать про незаконченное

Лучше сказать самому, чем ждать, пока найдут.

- **Время изготовления не вычисляется.** В конструкторе стоит постоянная надпись «2–3 weeks». В документе описана формула `T = Tb × C` — она пока не реализована, считается только цена.
- **Оплата работает в тестовом режиме Stripe.** Ключи `sk_test_` и вебхук подключены на Render; настоящие деньги не списываются. Для боевых платежей нужно переключить Stripe в live-режим и заменить ключи.
- **Цвет не меняет картинку в превью.** В конструкторе выбор цвета обновляет цену и образец, но большое изображение показывает фотографию размера или ремешка.